

model

April 14, 2025

** #

Import Libraries

Table of Contents

```
[2]: import os
import cv2
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import keras
from keras.callbacks import EarlyStopping, ModelCheckpoint
import tensorflow as tf
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from tqdm import tqdm
```

** #

Read Data

Table of Contents

```
[3]: def data(dataset_path):
    images = []
    labels = []
    for subfolder in tqdm(os.listdir(dataset_path)):
        subfolder_path = os.path.join(dataset_path, subfolder)
        for image_filename in os.listdir(subfolder_path):
            image_path = os.path.join(subfolder_path, image_filename)
            images.append(image_path)
            labels.append(subfolder)
    df = pd.DataFrame({'image': images, 'label': labels})
    return df
```

```
[4]: !pip install kaggle
import json # Import the json module
kaggle_credentails = json.load(open("kaggle.json"))
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.10/dist-packages
(1.6.17)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.10/dist-
packages (from kaggle) (1.16.0)
Requirement already satisfied: certifi>=2023.7.22 in
/usr/local/lib/python3.10/dist-packages (from kaggle) (2024.8.30)
Requirement already satisfied: python-dateutil in
/usr/local/lib/python3.10/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-
packages (from kaggle) (2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages
(from kaggle) (4.66.6)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.10/dist-
packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-
packages (from kaggle) (2.2.3)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages
(from kaggle) (6.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-
packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode>=1.3 in
/usr/local/lib/python3.10/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-
packages (from requests->kaggle) (3.10)
```

```
[5]: import os
from zipfile import ZipFile # Import the ZipFile class from the zipfile module
from tqdm import tqdm
import pandas as pd
# setup Kaggle API key as environment variables
kaggle_credentails = json.load(open("kaggle.json"))

os.environ['KAGGLE_USERNAME'] = kaggle_credentails["username"]
os.environ['KAGGLE_KEY'] = kaggle_credentails["key"]

!kaggle datasets download -d abdallahalidev/plantvillage-dataset

!ls
# Unzip the downloaded dataset
with ZipFile("plantvillage-dataset.zip", 'r') as zip_ref:
    zip_ref.extractall()
```

Dataset URL: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>
 License(s): CC-BY-NC-SA-4.0
 Downloading plantvillage-dataset.zip to /content
 100% 2.03G/2.04G [00:22<00:00, 114MB/s]
 100% 2.04G/2.04G [00:22<00:00, 99.2MB/s]
 kaggle.json plantvillage-dataset.zip sample_data

```
[6]: #train
train = data('/content/plantvillage dataset/color')
train.head()
```

100%| | 38/38 [00:00<00:00, 434.32it/s]

```
[6]:                                     image          label
0  /content/plantvillage dataset/color/Tomato___L...  Tomato___Leaf_Mold
1  /content/plantvillage dataset/color/Tomato___L...  Tomato___Leaf_Mold
2  /content/plantvillage dataset/color/Tomato___L...  Tomato___Leaf_Mold
3  /content/plantvillage dataset/color/Tomato___L...  Tomato___Leaf_Mold
4  /content/plantvillage dataset/color/Tomato___L...  Tomato___Leaf_Mold
```

```
[7]: train.shape
```

```
[7]: (54305, 2)
```

```
** #
```

Visualization of Dataset

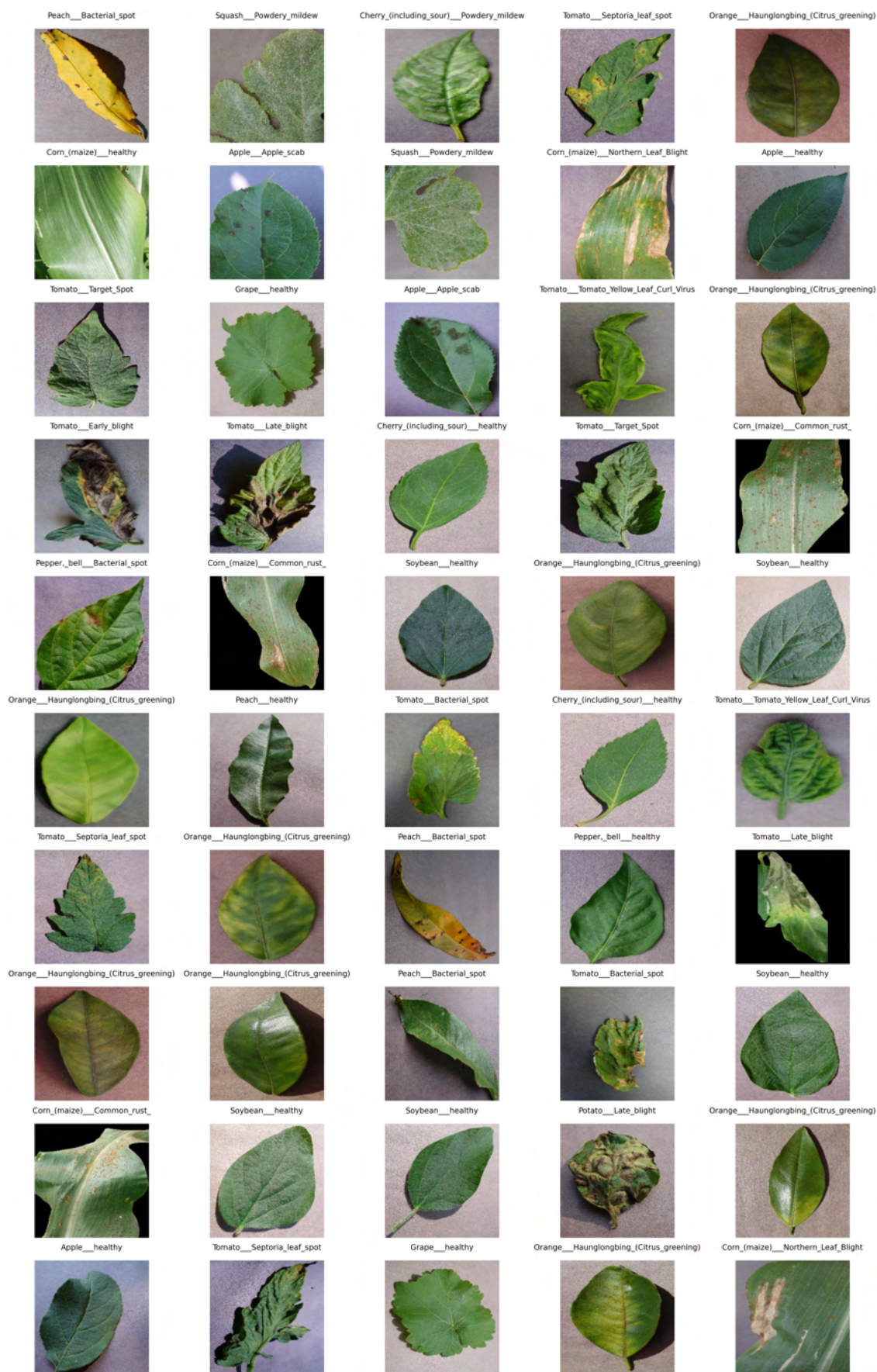
Table of Contents

```
[8]: train.label.value_counts().to_frame()
```

```
[8]:                                     count
label
Orange___Haunglongbing_(Citrus_greening)    5507
Tomato___Tomato_Yellow_Leaf_Curl_Virus      5357
Soybean___healthy                          5090
Peach___Bacterial_spot                     2297
Tomato___Bacterial_spot                     2127
Tomato___Late_blight                       1909
Squash___Powdery_mildew                    1835
Tomato___Septoria_leaf_spot                 1771
Tomato___Spider_mites_Two-spotted_spider_mite 1676
Apple___healthy                            1645
Tomato___healthy                           1591
Blueberry___healthy                        1502
Pepper,_bell___healthy                     1478
Tomato___Target_Spot                       1404
Grape___Esca_(Black_Measles)               1383
```

Corn_(maize)___Common_rust_	1192
Grape___Black_rot	1180
Corn_(maize)___healthy	1162
Strawberry___Leaf_scorch	1109
Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	1076
Cherry_(including_sour)___Powdery_mildew	1052
Potato___Early_blight	1000
Tomato___Early_blight	1000
Potato___Late_blight	1000
Pepper,_bell___Bacterial_spot	997
Corn_(maize)___Northern_Leaf_Blight	985
Tomato___Leaf_Mold	952
Cherry_(including_sour)___healthy	854
Apple___Apple_scab	630
Apple___Black_rot	621
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	513
Strawberry___healthy	456
Grape___healthy	423
Tomato___Tomato_mosaic_virus	373
Raspberry___healthy	371
Peach___healthy	360
Apple___Cedar_apple_rust	275
Potato___healthy	152

```
[72]: plt.style.use('default')
plt.figure(figsize=(50,80))
for n,i in enumerate(np.random.randint(0,len(train),50)):
    plt.subplot(10,5,n+1)
    img = cv2.imread(train.image[i])
    img = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
    plt.imshow(img)
    plt.axis('off')
    text = f'{train.label[i]}\n'
    plt.title(text,fontsize=25)
```



#

Data Augmentation

Table of Contents

```
[10]: X_train, X_test1, y_train, y_test1 = train_test_split(train['image'],  
    ↪train['label'], test_size=0.2,  
    ↪random_state=42,shuffle=True,stratify=train['label'])  
X_val, X_test, y_val, y_test = train_test_split(X_test1,y_test1, test_size=0.5,  
    ↪random_state=42,shuffle=True,stratify=y_test1)  
df_train = pd.DataFrame({'image': X_train, 'label': y_train})  
df_test = pd.DataFrame({'image': X_test, 'label': y_test})  
df_val = pd.DataFrame({'image': X_val, 'label': y_val})
```

```
[11]: image_size = (224, 224)  
batch_size = 32  
datagen = ImageDataGenerator(  
    rescale=1./255  
)  
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    #rotation_range=20,  
    #width_shift_range=0.2,  
    #height_shift_range=0.2,  
    #shear_range=0.2,  
    #zoom_range=0.2,  
    horizontal_flip=True,  
    #fill_mode='nearest'  
)  
train_generator = train_datagen.flow_from_dataframe(  
    df_train,  
    x_col='image',  
    y_col='label',  
    target_size=image_size,  
    batch_size=batch_size,  
    shuffle=True  
)  
test_generator = datagen.flow_from_dataframe(  
    df_test,  
    x_col='image',  
    y_col='label',  
    target_size=image_size,  
    batch_size=batch_size,  
    shuffle=False  
)
```

```

val_generator = datagen.flow_from_dataframe(
    df_val,
    x_col='image',
    y_col='label',
    target_size=image_size,
    batch_size=batch_size,
    shuffle=False
)

```

Found 43444 validated image filenames belonging to 38 classes.

Found 5431 validated image filenames belonging to 38 classes.

Found 5430 validated image filenames belonging to 38 classes.

```

[12]: class_ = test_generator.class_indices.keys()
      class_ = list(class_)

```

#

Modeling

Tabel of Contents

```

[13]: model = keras.models.Sequential([
        keras.layers.Conv2D(32, kernel_size = (3,3), activation = 'relu',
        ↪input_shape = (224,224,3)),
        keras.layers.MaxPooling2D(2,2),

        keras.layers.Conv2D(64, kernel_size = (3,3), activation = 'relu'),
        keras.layers.MaxPooling2D(2,2),

        keras.layers.Conv2D(64, kernel_size = (3,3), activation = 'relu'),
        keras.layers.MaxPooling2D(2,2),

        keras.layers.Conv2D(64, kernel_size = (3,3), activation = 'relu'),
        keras.layers.MaxPooling2D(2,2),

        keras.layers.Conv2D(64, kernel_size = (3,3), activation = 'relu'),
        keras.layers.MaxPooling2D(2,2),

        keras.layers.Flatten(),

        keras.layers.Dense(64, activation='relu'),

        keras.layers.Dense(38, activation='softmax'),
    ])

```

```
]
model.summary()
```

```
/usr/local/lib/python3.10/dist-
packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
```

```
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "sequential"
```

Layer (type)	Output Shape	
↳Param #		
conv2d (Conv2D)	(None, 222, 222, 32)	
↳896		
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	
↳ 0		
conv2d_1 (Conv2D)	(None, 109, 109, 64)	
↳18,496		
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	
↳ 0		
conv2d_2 (Conv2D)	(None, 52, 52, 64)	
↳36,928		
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 64)	
↳ 0		
conv2d_3 (Conv2D)	(None, 24, 24, 64)	
↳36,928		
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 64)	
↳ 0		
conv2d_4 (Conv2D)	(None, 10, 10, 64)	
↳36,928		
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 64)	
↳ 0		

```

conv2d_5 (Conv2D)                (None, 3, 3, 64)
↳36,928

max_pooling2d_5 (MaxPooling2D)   (None, 1, 1, 64)
↳ 0

flatten (Flatten)                (None, 64)
↳ 0

dense (Dense)                    (None, 64)
↳4,160

dense_1 (Dense)                  (None, 38)
↳2,470

```

Total params: 173,734 (678.65 KB)

Trainable params: 173,734 (678.65 KB)

Non-trainable params: 0 (0.00 B)

```

[14]: checkpoint_cb =ModelCheckpoint("my_keras_model.keras", save_best_only=True)
early_stopping_cb =EarlyStopping(patience=5, restore_best_weights=True)
model.compile(optimizer ='adam', loss='categorical_crossentropy',
↳metrics=['accuracy',keras.metrics.AUC()])
hist=model.
↳fit(train_generator,epochs=5,validation_data=val_generator,callbacks=[checkpoint_cb,early_s

```

Epoch 1/5

```

/usr/local/lib/python3.10/dist-
packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.

```

```

    self._warn_if_super_not_called()

```

```

1358/1358          148s 100ms/step
- accuracy: 0.3068 - auc: 0.8380 - loss: 2.5507 - val_accuracy: 0.6711 -
val_auc: 0.9772 - val_loss: 1.0784

```

Epoch 2/5

```

1358/1358          110s 81ms/step
- accuracy: 0.7128 - auc: 0.9820 - loss: 0.9250 - val_accuracy: 0.7744 -
val_auc: 0.9884 - val_loss: 0.7253

```

```
Epoch 3/5
1358/1358          143s 81ms/step
- accuracy: 0.8088 - auc: 0.9910 - loss: 0.6029 - val_accuracy: 0.8319 -
val_auc: 0.9921 - val_loss: 0.5294
Epoch 4/5
1358/1358          105s 77ms/step
- accuracy: 0.8540 - auc: 0.9940 - loss: 0.4549 - val_accuracy: 0.8790 -
val_auc: 0.9957 - val_loss: 0.3726
Epoch 5/5
1358/1358          102s 75ms/step
- accuracy: 0.8780 - auc: 0.9954 - loss: 0.3763 - val_accuracy: 0.8783 -
val_auc: 0.9950 - val_loss: 0.3586
```

#

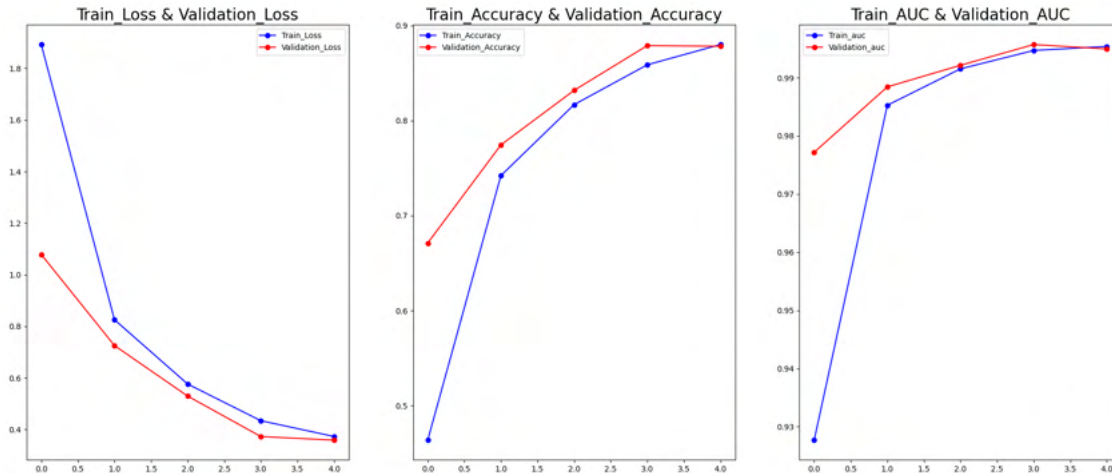
Evaluate The Model

Table of Contents

```
[15]: hist=pd.DataFrame(hist.history)
hist_
```

```
[15]:      accuracy      auc      loss  val_accuracy  val_auc  val_loss
0  0.463539  0.927683  1.892239      0.671087  0.977181  1.078368
1  0.742105  0.985270  0.825570      0.774401  0.988426  0.725333
2  0.816799  0.991535  0.576157      0.831860  0.992147  0.529447
3  0.858784  0.994689  0.433999      0.879006  0.995709  0.372627
4  0.880099  0.995388  0.372859      0.878269  0.994952  0.358628
```

```
[73]: plt.style.use('default')
plt.figure(figsize=(25,10))
plt.subplot(1,3,1)
plt.plot(hist_['loss'],'b-o',label='Train_Loss')
plt.plot(hist_['val_loss'],'r-o',label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,3,2)
plt.plot(hist_['accuracy'],'b-o',label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],'r-o',label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.subplot(1,3,3)
plt.plot(hist_.iloc[:,1],'b-o',label='Train_auc')
plt.plot(hist_.iloc[:,4],'r-o',label='Validation_auc')
plt.title('Train_AUC & Validation_AUC',fontsize=20)
plt.legend()
plt.show()
```



```
[17]: score, acc, auc = model.evaluate(test_generator)
print('Test Loss =', score)
print('Test Accuracy =', acc)
print('Test AUC =', auc)
```

2/170 8s 52ms/step - accuracy:

0.8750 - auc: 0.9990 - loss: 0.2746

/usr/local/lib/python3.10/dist-

packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121:

UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

self._warn_if_super_not_called()

170/170 13s 77ms/step -

accuracy: 0.8798 - auc: 0.9943 - loss: 0.3711

Test Loss = 0.3719049394130707

Test Accuracy = 0.8814214468002319

Test AUC = 0.9945454001426697

```
[18]: # Get true labels
y_test = test_generator.classes
# Predict probabilities for each class
predictions = model.predict(test_generator)
# Convert predicted probabilities to class labels (choose the class with the
# highest probability)
y_pred = np.argmax(predictions, axis=1)
# Flatten both arrays just in case
y_test = np.ravel(y_test)
y_pred = np.ravel(y_pred)
```

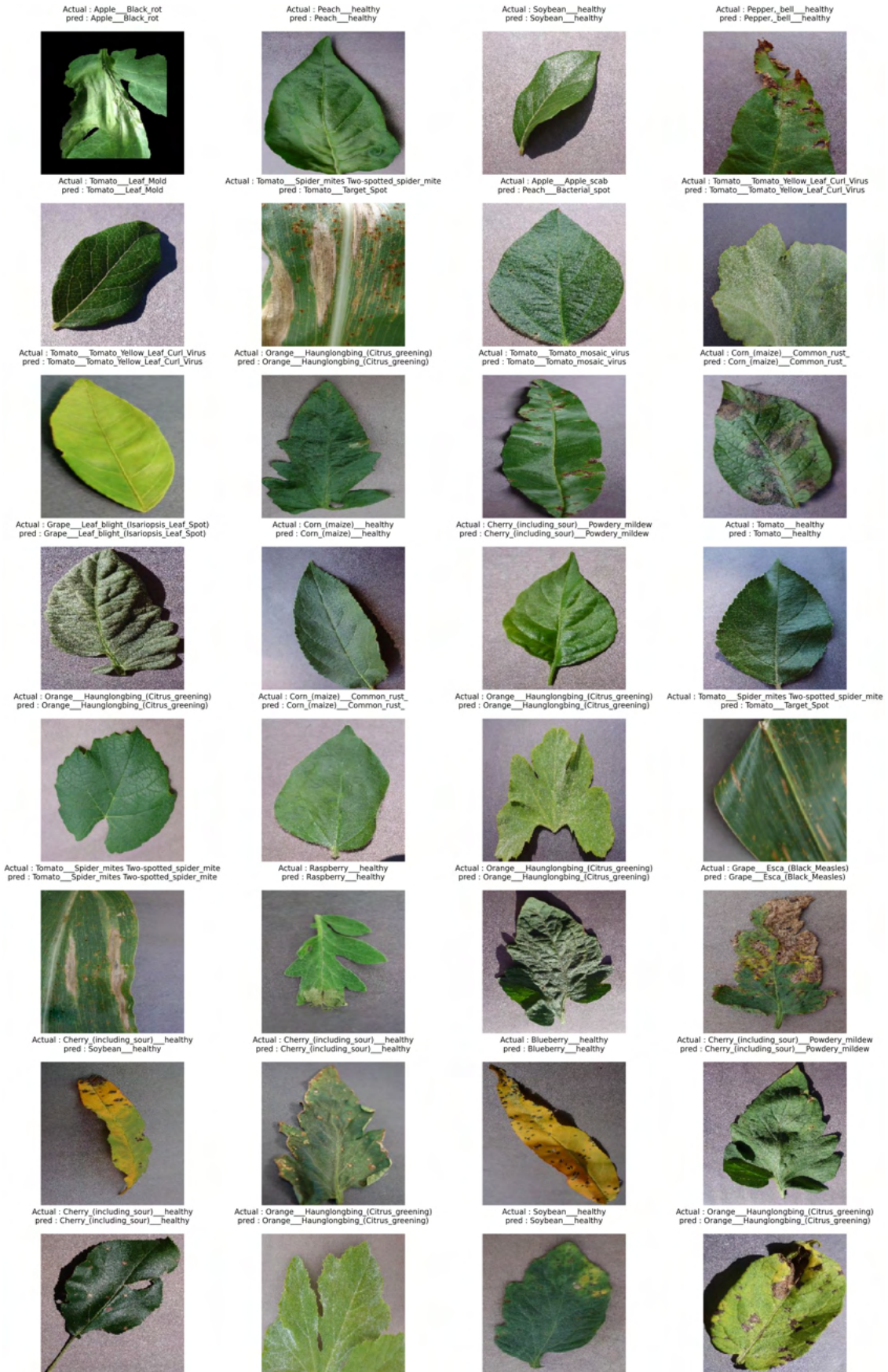
```
# Create a DataFrame to compare actual vs predicted labels
df = pd.DataFrame({'Actual': y_test, 'Prediction': y_pred})
# Display the DataFrame
df.head() # Show the first few rows of the comparison
```

170/170 10s 54ms/step

```
[18]:
```

	Actual	Prediction
0	1	1
1	17	17
2	24	24
3	19	19
4	31	31

```
[74]: batch = next(test_generator)
images= batch[0]
plt.figure(figsize=(50,80))
for n in range(32):
    plt.subplot(8,4,n+1)
    plt.imshow(images[n])
    plt.axis('off')
    text = f'Actual : {class_[int(y_test[n])]} \npred : 
↪ {class_[int(y_pred[n])]} \n'
    plt.title(text,fontsize=25)
    plt.style.use('default')
```



```
[76]: CM = confusion_matrix(y_test,y_pred)
plt.figure(figsize=(15,25))
plt.style.use('default')

sns.heatmap(CM,fmt='g',center = 0,
↪ True,cbar=False,annot=True,cmap='Set2',xticklabels=class_,yticklabels=class_)
CM
```

```
[76]: array([[ 42,   0,   0, ...,   0,   0,   0],
[   0,  55,   0, ...,   0,   0,   0],
[   0,   0,  21, ...,   0,   0,   0],
...,
[   0,   0,   3, ..., 505,   0,   0],
[   0,   0,   0, ...,   0,  34,   0],
[   0,   0,   0, ...,   0,   0, 145]])
```

15

```
[21]: ClassificationReport = classification_report(y_test,y_pred,target_names=class_)
print('Classification Report is : ', ClassificationReport)
```

Classification Report is :

precision	recall	f1-score	support			
				Apple___Apple_scab	0.56	0.67
0.61	63			Apple___Black_rot	0.92	0.89
0.90	62			Apple___Cedar_apple_rust	0.70	0.78
0.74	27			Apple___healthy	0.86	0.81
0.83	165			Blueberry___healthy	0.72	0.98
0.83	150			Cherry_(including_sour)___Powdery_mildew	0.82	0.96
0.89	105			Cherry_(including_sour)___healthy	0.87	0.92
0.89	85			Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	0.75	0.65
0.69	51			Corn_(maize)___Common_rust_	0.91	0.97
0.94	120			Corn_(maize)___Northern_Leaf_Blight	0.81	0.86
0.83	98			Corn_(maize)___healthy	0.97	0.96
0.97	116			Grape___Black_rot	0.86	0.92
0.89	118			Grape___Esca_(Black_Measles)	0.88	0.98
0.93	139			Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	0.98	0.87
0.92	108			Grape___healthy	0.87	0.93
0.90	42			Orange___Haunglongbing_(Citrus_greening)	0.97	0.99
0.98	551			Peach___Bacterial_spot	0.78	0.94
0.85	230			Peach___healthy	0.68	1.00
0.81	36			Pepper,_bell___Bacterial_spot	0.91	0.70
0.79	99			Pepper,_bell___healthy	1.00	0.63
0.77	148					

		Potato___Early_blight	0.92	0.93
0.93	100			
		Potato___Late_blight	0.67	0.81
0.73	100			
		Potato___healthy	0.71	0.67
0.69	15			
		Raspberry___healthy	0.80	0.89
0.85	37			
		Soybean___healthy	0.95	0.96
0.96	509			
		Squash___Powdery_mildew	0.99	0.85
0.91	184			
		Strawberry___Leaf_scorch	0.93	0.98
0.96	111			
		Strawberry___healthy	0.92	0.98
0.95	45			
		Tomato___Bacterial_spot	0.85	0.93
0.89	213			
		Tomato___Early_blight	0.83	0.48
0.61	100			
		Tomato___Late_blight	0.87	0.70
0.77	191			
		Tomato___Leaf_Mold	0.90	0.78
0.84	95			
		Tomato___Septoria_leaf_spot	0.87	0.59
0.70	177			
		Tomato___Spider_mites Two-spotted_spider_mite	0.83	0.92
0.88	168			
		Tomato___Target_Spot	0.71	0.84
0.77	141			
		Tomato___Tomato_Yellow_Leaf_Curl_Virus	0.99	0.94
0.96	536			
		Tomato___Tomato_mosaic_virus	0.68	0.92
0.78	37			
		Tomato___healthy	0.99	0.91
0.95	159			
		accuracy		
0.88	5431			
		macro avg	0.85	0.85
0.84	5431			
		weighted avg	0.89	0.88
0.88	5431			

```
[65]: dtm = model.save('my_keras_model.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or

`keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

[77]: `!pip install Pillow`

```
import numpy as np
from PIL import Image

def load_and_preprocess_image(image_path, target_size=(224, 224)):

    try:
        # Load the image using PIL's Image
        img = Image.open(image_path)
        # Resize the image
        img = img.resize(target_size)
        # Convert the image to RGB mode if it's not already
        if img.mode != 'RGB':
            img = img.convert('RGB')
        # Convert the image to a NumPy array and normalize pixel values
        img_array = np.array(img) / 255.0

        # Add an extra dimension to represent the batch size (for Keras)
        img_array = np.expand_dims(img_array, axis=0)
        return img_array
    except (FileNotFoundError, PIL.UnidentifiedImageError, OSError) as e:
        print(f"Error loading or preprocessing image: {e}")
        return None

def predict_image_class(model, image_path, class_indices):

    preprocessed_img = load_and_preprocess_image(image_path)

    # Check if image loading was successful
    if preprocessed_img is None:
        return "Error: Image could not be loaded or preprocessed."

    predictions = model.predict(preprocessed_img)
    predicted_class_index = np.argmax(predictions, axis=1)[0]
    predicted_class_name = class_indices[predicted_class_index]
    return predicted_class_name

# ... (Your code for loading the model, class_indices, etc.)

image_path = "/content/plantvillage dataset/color/Apple__Black_rot/
↳0199a733-6d3f-4e66-a903-2d1d34457871__JR_FrgE.S 3079.JPG"
```

```
# ... (Other image paths)

predicted_class_name = predict_image_class(model, image_path, class_indices)

# Output the result
print("Predicted Class Name:", predicted_class_name)
```

```
Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages
(10.4.0)
1/1          0s 270ms/step
Predicted Class Name: Apple__Black_rot
```

[78]: `!pip install lime`

```
Requirement already satisfied: lime in /usr/local/lib/python3.10/dist-packages
(0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-
packages (from lime) (3.8.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages
(from lime) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages
(from lime) (1.13.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages
(from lime) (4.66.6)
Requirement already satisfied: scikit-learn>=0.18 in
/usr/local/lib/python3.10/dist-packages (from lime) (1.5.2)
Requirement already satisfied: scikit-image>=0.12 in
/usr/local/lib/python3.10/dist-packages (from lime) (0.24.0)
Requirement already satisfied: networkx>=2.8 in /usr/local/lib/python3.10/dist-
packages (from scikit-image>=0.12->lime) (3.4.2)
Requirement already satisfied: pillow>=9.1 in /usr/local/lib/python3.10/dist-
packages (from scikit-image>=0.12->lime) (10.4.0)
Requirement already satisfied: imageio>=2.33 in /usr/local/lib/python3.10/dist-
packages (from scikit-image>=0.12->lime) (2.36.0)
Requirement already satisfied: tifffile>=2022.8.12 in
/usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime)
(2024.9.20)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.10/dist-
packages (from scikit-image>=0.12->lime) (24.1)
Requirement already satisfied: lazy-loader>=0.4 in
/usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-
packages (from scikit-learn>=0.18->lime) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.18->lime) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib->lime) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-
```

```

packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib->lime) (4.54.1)
Requirement already satisfied: kiwisolver>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib->lime) (1.4.7)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib->lime) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.10/dist-packages (from matplotlib->lime) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-
packages (from python-dateutil>=2.7->matplotlib->lime) (1.16.0)

```

```

[79]: import lime
import numpy as np
import cv2
from lime import lime_image
from skimage.segmentation import mark_boundaries
import matplotlib.pyplot as plt

# Load the trained model
from keras.models import load_model
model = load_model('my_keras_model.h5')

```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

```

[80]: # Define a function to preprocess the image for LIME
def preprocess_image(image_path, target_size=(224, 224)):
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = cv2.resize(image, target_size)
    image = image / 255.0 # Normalize the image
    return image

# Choose an image for explanation
# Choose the first test image
image = preprocess_image(image_path)

# LIME requires a batch of images
image_batch = np.expand_dims(image, axis=0)

# Create a LIME explainer
explainer = lime_image.LimeImageExplainer()

# Define a function that returns model predictions for LIME
def predict_fn(images):

```

```

    return model.predict(images)

# Explain the image prediction
explanation = explainer.explain_instance(
    image,
    predict_fn,
    top_labels=3,
    hide_color=0,
    num_samples=1000 # Number of perturbations
)

# Display the explanation for the top class
top_label = explanation.top_labels[0]
temp, mask = explanation.get_image_and_mask(
    top_label,
    positive_only=False,
    num_features=5,
    hide_rest=False
)

```

```

0%|          | 0/1000 [00:00<?, ?it/s]

1/1          0s 289ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 23ms/step
1/1          0s 20ms/step
1/1          0s 19ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 24ms/step
1/1          0s 24ms/step
1/1          0s 25ms/step
1/1          0s 21ms/step
1/1          0s 21ms/step
1/1          0s 20ms/step
1/1          0s 25ms/step
1/1          0s 22ms/step
1/1          0s 25ms/step
1/1          0s 22ms/step
1/1          0s 24ms/step
1/1          0s 25ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 20ms/step
1/1          0s 21ms/step

```

1/1	0s 21ms/step
1/1	0s 21ms/step
1/1	0s 24ms/step
1/1	0s 23ms/step
1/1	0s 20ms/step
1/1	0s 29ms/step
1/1	0s 30ms/step
1/1	0s 27ms/step
1/1	0s 27ms/step
1/1	0s 42ms/step
1/1	0s 44ms/step
1/1	0s 26ms/step
1/1	0s 29ms/step
1/1	0s 26ms/step
1/1	0s 30ms/step
1/1	0s 30ms/step
1/1	0s 38ms/step
1/1	0s 37ms/step
1/1	0s 30ms/step
1/1	0s 28ms/step
1/1	0s 28ms/step
1/1	0s 35ms/step
1/1	0s 36ms/step
1/1	0s 32ms/step
1/1	0s 34ms/step
1/1	0s 33ms/step
1/1	0s 20ms/step
1/1	0s 20ms/step
1/1	0s 19ms/step
1/1	0s 27ms/step
1/1	0s 19ms/step
1/1	0s 19ms/step
1/1	0s 44ms/step
1/1	0s 20ms/step
1/1	0s 20ms/step
1/1	0s 20ms/step
1/1	0s 21ms/step
1/1	0s 19ms/step
1/1	0s 19ms/step
1/1	0s 19ms/step
1/1	0s 19ms/step
1/1	0s 23ms/step
1/1	0s 35ms/step
1/1	0s 19ms/step
1/1	0s 20ms/step
1/1	0s 22ms/step
1/1	0s 22ms/step
1/1	0s 20ms/step

1/1	0s 20ms/step
1/1	0s 19ms/step
1/1	0s 25ms/step
1/1	0s 26ms/step
1/1	0s 29ms/step
1/1	0s 20ms/step
1/1	0s 19ms/step
1/1	0s 23ms/step
1/1	0s 20ms/step
1/1	0s 21ms/step
1/1	0s 21ms/step
1/1	0s 20ms/step
1/1	0s 29ms/step
1/1	0s 25ms/step
1/1	0s 20ms/step
1/1	0s 20ms/step
1/1	0s 19ms/step
1/1	0s 19ms/step
1/1	0s 20ms/step
1/1	0s 20ms/step
1/1	0s 21ms/step
1/1	0s 25ms/step
1/1	0s 25ms/step
1/1	0s 21ms/step
1/1	0s 19ms/step
1/1	0s 22ms/step

```
[81]: # Plot the original image and the LIME explanation
plt.style.use('default')

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(mark_boundaries(temp, mask))
plt.title("LIME Explanation")
plt.axis('off')

plt.show()
```

Original Image



LIME Explanation

